

Bitcask

用于快速键/值数据的日志结构哈希表

Justin Sheehy, David Smith,
灵感来源于 Eric Brewer

Basho Technologies
justin@basho.com,dizzyd@basho.com

Bitcask 的起源与 Riak 分布式数据库的历史紧密相关。在 Riak 键值集群中，每个节点均采用可插拔的本地存储；几乎所有键值形态的数据结构都可作为每台主机的存储引擎。这种可插拔性使得 Riak 的开发能够并行推进，存储引擎的改进与测试可在不影响其余代码库的前提下进行。

许多这样的本地键值存储已经存在，包括但不限于 Berkeley DB、Tokyo Cabinet 和 Innostore。在评估这类存储引擎时，我们追求多个目标，包括：

- 每项读写操作的低延迟
- 高吞吐量，尤其在写入随机数据流时。
- 能够处理远大于 RAM 的数据集而不会出现性能下降
- 崩溃友好性，体现在快速恢复和不丢失数据两方面
- 备份与恢复的便捷性
- 相对简单、易于理解（从而可维护）的代码结构与数据格式

在高访问负载或大容量下可预测的行为

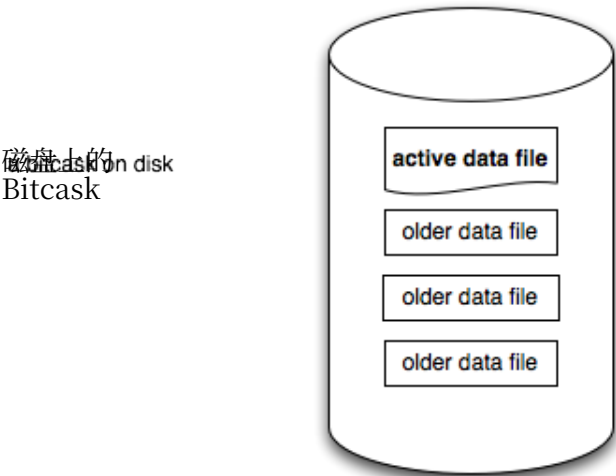
- 一种在 Riak 中易于默认使用的许可证

实现其中一些较为容易，但全部实现则难度更大。

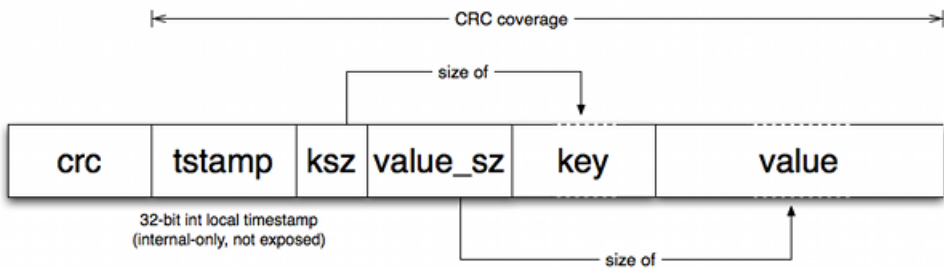
现有的本地键值存储系统（包括但不限于本文作者所开发的系统）均未能完全满足上述所有目标。在与 Eric Brewer 讨论这一问题时，他提出了关于哈希表日志合并的关键见解：即该操作有潜力达到与 LSM 树相当甚至更快的速度。

这使得我们重新审视了最初在 20 世纪 80 年代和 90 年代开发的日志结构文件系统中使用的若干技术，并由此催生了 Bitcask——一种能很好地满足上述所有目标的存储系统。尽管 Bitcask 最初是为了在 Riak 中使用而开发的，但其设计具有通用性，同样可作为其他应用程序的本地键值存储。最终采用的模型在概

念上非常简洁：一个 Bitcask 实例就是一个目录，且我们强制规定在任意时刻只有一个操作系统进程能打开该 Bitcask 进行写入——你可以将该进程视为“数据库服务器”。在任何时刻，该目录中只有一个文件处于“活跃”状态，供服务器写入。当该文件达到大小阈值时，它将被关闭，并创建新的活跃文件。一旦文件被关闭（无论是主动关闭还是因服务器退出而关闭），它便被视作不可变的，永远不会再被打开写入。



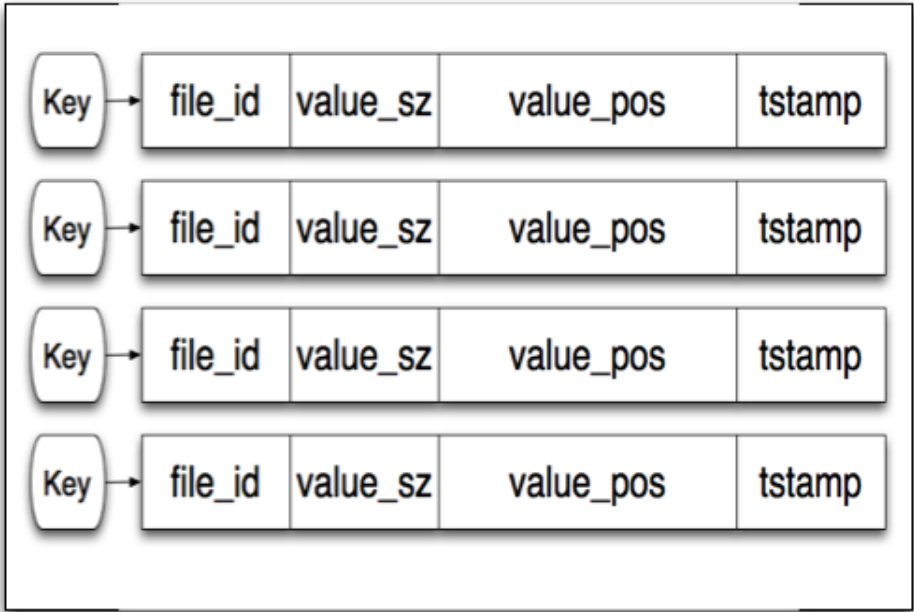
活跃文件仅通过追加方式写入，这意味着顺序写入无需磁盘寻道。每个键/值条目的写入格式简单：



每次写入操作都会向活动文件追加一个新条目。需要注意的是，删除操作本质上也是写入一个特殊的墓碑值，该值将在下一次合并时被移除。因此，一个 Bitcask 数据文件实际上就是这些条目的线性序列：

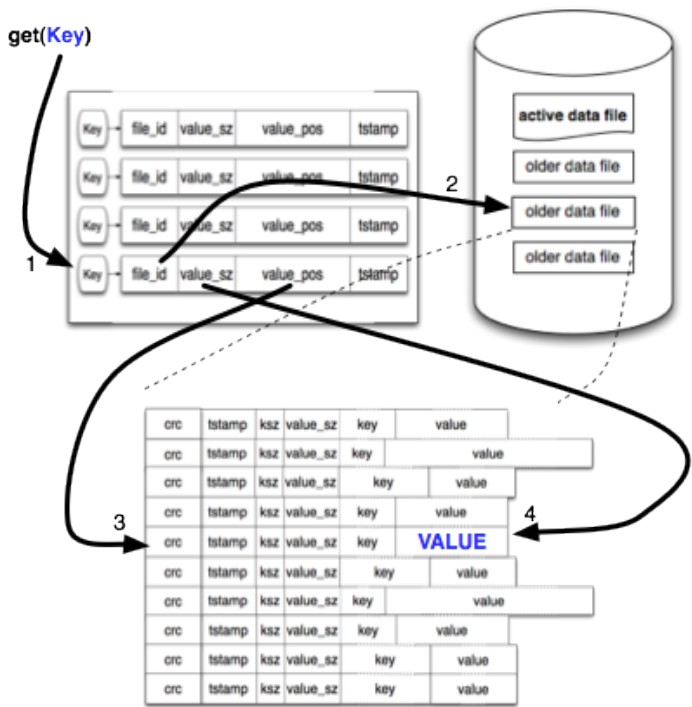
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value

追加写入完成后，会更新一个名为“keydir”的内存结构。keydir 本质上是一个哈希表，它将 Bitcask 中的每个键映射到一个固定大小的结构，该结构包含该键最新写入条目的文件、偏移量和大小。



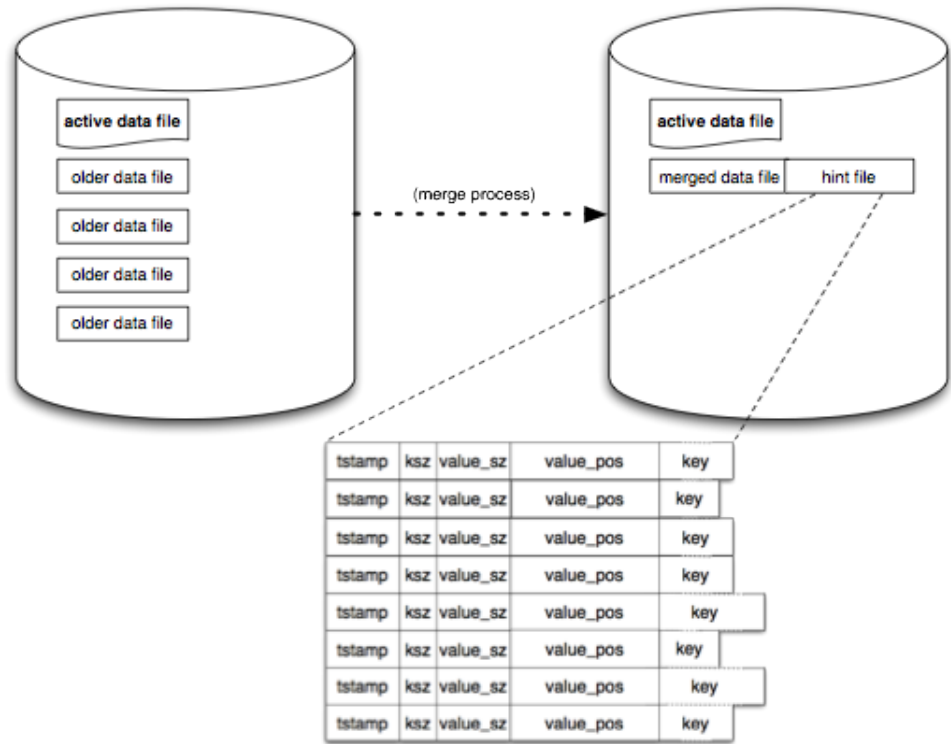
当发生写入时，keydir 会被原子地更新为最新数据的位置。旧数据仍然存在于磁盘上，但所有的读取操作将使用 keydir 中提供的最新版本。如后文所述，合并过程最终会移除旧值。

读取值的操作非常简单，且最多只需一次磁盘寻道。我们在键目录（keydir）中查找键，然后根据查找返回的 file_id、position 和 size 读取数据。在许多情况下，操作系统的文件系统预读缓存使得这一操作的执行速度远比预期更快。



这种简单的模型随着时间的推移可能会占用大量空间，因为我们只是写入新值而不处理旧值。一种称之为“合并”的压缩过程可以解决这个问题。合并过程遍历 Bitcask 中所有非活动（即不可变）文件，并生成一组仅包含每个现有键的“存活”或最新版本的数据文件作为输出。

完成此操作后，我们还会在每个数据文件旁边创建一个“hint file”。这些文件本质上类似于数据文件，但其中包含的不是值本身，而是对应数据文件中值的位置和大小。



当 Erlang 进程打开一个 Bitcask 实例时，它会检查同一虚拟机内是否已有其他 Erlang 进程正在使用该 Bitcask。如果是，则与该进程共享 keydir；否则，它会扫描目录中的所有数据文件，以构建一个新的 keydir。对于任何附带 hint 文件的数据文件，将转而扫描该 hint 文件，从而大幅缩短启动时间。

这些基本操作是 bitcask 系统的核心。显然，本文并未试图详尽展示所有操作细节；我们的目标是帮助读者理解 Bitcask 的通用机制。对于文中一笔带过的几个方面，或许有必要作一些补充说明：

- 我们提到过，利用操作系统的文件系统缓存来实现读取性能。也曾讨论过增加一个 bitcask 内部的读取缓存，但鉴于目前已有的免费收益相当可观，尚不清楚这一改进能带来多大回报。

我们很快将给出与多种 API 类似的本地存储系统的基准测试结果。然而，Bitcask 的初始目标并非成为最快的存储引擎，而是追求“足够”的速度，同时保证代码、设计和文件格式的高质量与简洁。尽管如此，在我们初步的简单基准测试中，Bitcask 已在许多场景下轻松超越其他快速存储系统。

- 部分实现细节最为困难，但对外部人士而言也最无趣，因此本文档未对（例如）内部 keydir 锁定方案进行描述。
- Bitcask 不对数据执行任何压缩，因为压缩的成本/效益高度依赖于具体应用。

下面来看我们最初设定的目标：

- **低读写延迟**

Bitcask 速度很快。我们计划很快进行更全面的基准测试，但在早期测试中，典型中位延迟低于毫秒级（高百分位的表现也相当可观），这让我们相信它能够达到我们的速度目标。

- **高吞吐量，尤其在写入随机数据流时。**

在采用慢速磁盘的笔记本电脑的早期测试中，我们观察到每秒 5000-6000 次写入的吞吐量。

- **处理远大于 RAM 的数据集而无性能退化**

上述测试在所讨论的系统上使用了超过 $10 \times \text{RAM}$ 的数据集，且未观察到任何行为变化的迹象。这与我们基于 Bitcask 设计的预期一致。

- **崩溃友好性：快速恢复与数据不丢失**

在 Bitcask 中，数据文件与提交日志是同一实体，因此恢复过程十分简单，无需“回放”。提示文件可用于加速启动过程。

- **备份与恢复的便捷性**

由于文件在轮转后不可变，备份可轻松采用操作人员偏好的任何系统级机制，恢复只需将数据文件放入所需目录即可。

- **相对简单、易于理解（从而可维护）的代码结构与数据格式**

Bitcask 在概念上简单明了，代码简洁，数据文件非常易于理解和管理。我们对于支持基于 Bitcask 构建的系统感到十分放心。

- **高访问负载或大容量下的可预测行为**

在高访问负载下，我们已看到 Bitcask 表现良好。目前它仅处理了数十 GB 量级的数据，但我们很快将用更大规模的数据进行测试。Bitcask 的设计特性使其在更大数据量下预计不会出现显著性能差异，唯一可预见的例外是，keydir 结构会随键数量小幅增长，且必须完全驻留在 RAM 中。在实际应用中这一限制影响甚微，因为即便拥有数百万个键，当前实现所使用的内存也远低于 1 GB。

总之，在此特定目标集下，Bitcask 比我们拥有的任何其他方案都更符合需求。

该 API 非常简单:

bitcask:open(DirectoryName, Opts) → BitCaskHandle {error, any()}	Open a new or existing Bitcask datastore with additional options. Valid options include read_write (if this process is going to be a writer and not just a reader) and sync_on_put (if this writer would prefer to sync the write file after every write operation). The directory must be readable and writable by this process, and only one process may open a Bitcask with read_write at a time.
bitcask:open(DirectoryName) → BitCaskHandle {error, any()}	Open a new or existing Bitcask datastore for read-only access. The directory and all files in it must be readable by this process.
bitcask:get(BitCaskHandle, Key) → not_found {ok, Value}	Retrieve a value by key from a Bitcask datastore.
bitcask:put(BitCaskHandle, Key, Value) → ok {error, any()}	Store a key and value in a Bitcask datastore.
bitcask:delete(BitCaskHandle, Key) → ok {error, any()}	Delete a key from a Bitcask datastore.
bitcask:list_keys(BitCaskHandle) → [Key] {error, any()}	List all keys in a Bitcask datastore.
bitcask:fold(BitCaskHandle, Fun, Acc0) → Acc	Fold over all K/V pairs in a Bitcask datastore. Fun is expected to be of the form: F(K,V,Acc0) → Acc.
bitcask:merge(DirectoryName) → ok {error, any()}	Merge several data files within a Bitcask datastore into a more compact form. Also, produce hintfiles for faster startup.
bitcask:sync(BitCaskHandle) → ok	Force any writes to sync to disk.
bitcask:close(BitCaskHandle) → ok	Close a Bitcask data store and flush all pending writes (if any) to disk.

由此可见, Bitcask 使用起来非常简单。祝使用愉快!